

Defeasible Library Learning: Typed Methods with Runtime Contracts that Un-learn on Drift

Nathanael Braun

2026

Preprint v1 — system / experience report

Abstract

LLM agents reuse past work through *fuzzy* memory — retrieval (RAG), case-based reasoning (CBR), and prose skill libraries — which recalls by surface similarity and has no notion of a premise becoming false. When the world drifts in a way that does not alter the query itself, these stores keep serving a *stale* answer. We present **defeasible library learning**: a learned library of typed, composable *methods*, each carrying a **defeasible runtime contract**. A method is assumed at compose time, **asserted at run time**, and — when its induced postcondition fails — **retracted with blame** (a JTMS un-learning), after which the library is surgically revised rather than discarded. The same typed structure that makes a method canonicalizable also makes its reuse *amortizable* and its composition *checkable on contracts alone*, with bounded per-call context. We evaluate the claims on a real rule-driven, JTMS-coherent engine, isolating each mechanism (deterministic stub for the model, plus one live-model confirmation). Findings: (E2) under an external mid-stream premise-invalidation, recall-only memories serve **stale** (0 of the drift cases), whereas *any* cache with an invalidation hook recovers — what a *declarative typed contract* adds over a hand-coded invalidation callback is *selective*, principled eviction (re-assert the post; evict only what is violated), plus generality and composition-safety; all at bounded per-call context, confirmed on a live local model. (E1) cross-problem **structural transfer** is sound and free on the held-out related instances while the no-transform ablation is unsound — and call-count alone cannot tell them apart, only the soundness check can. (E3) a box-closed composition check matches open-the-box reality on every evaluated pair (no false-admits), and each of three soundness gates is load-bearing. (P4) amortization is a **gradient** in the canonicalizable fraction, soundness holds at every coverage, and amortizing *beyond* that fraction is unsound by construction. (E6) in a head-to-head against the *named* agent-memory systems (MemGPT, Reflexion, GraphRAG), each — given its fairest shot — can recover on drift, but only the typed contract recovers at the lowest cost on (calls $\times$ correctness $\times$ per-call context) *simultaneously*: the unique Pareto-optimal point, the others paying a paging / per-record / batch-re-index tax (stub + live). No mechanism here is new (JTMS, contracts-with-blame, library learning, theory revision, separation-logic footprints); the contribution is their **composition** into a learned method library that performs *principled, selective un-learning on drift*, which recall-only agent memories do not — bounded by a measured K1 ceiling.

1 Introduction

An agent that solves many related problems should get cheaper and more reliable over time. The dominant approach is to remember and recall: store past solutions or skills, retrieve the nearest for a new case, reuse or adapt. Retrieval-augmented generation [16], case-based reasoning, and

prose skill libraries such as Voyager [27] share this shape and a blind spot: they recall by *surface* similarity and have no representation of a *premise that has become false*. When the world changes in a way that does **not** change the query — a regulation tightened, a fact audited and found wrong, a policy revoked — the cached answer is still the nearest neighbour, and is still served. The memory is confidently stale. Static libraries (learned programs, distilled skills [11, 4]) have the opposite problem: sound when learned, but they cannot *un-learn*.

We argue the missing ingredient is a **defeasible typed contract** on each reusable unit. Borrowing from software contracts with blame [14] and gradual verification [3], a method declares what it reads, writes, requires, and guarantees over a *typed* fact alphabet. The guarantee of a *learned* method is an induced hypothesis, so it is **assumed** at compose time, **asserted** at run time, and **retracted with blame** when violated — a truth-maintenance operation [8, 7] — after which the library is revised by *specializing* the failed precondition, not by deletion (theory revision [19, 23]; belief revision [1]). The same typed structure makes reuse **amortize** (a stable canonical key) and composition **checkable without opening the box** (post of one entails pre of the next over the shared typed footprint [18, 22]). This power is bounded by a single honest constraint, K1: only typed, canonicalizable structure amortizes; genuine prose stays in the model.

We use *library learning* in the established sense of DreamCoder/Stitch [11, 4] (inducing reusable typed methods by abstraction [21]), not statistical parameter fitting; one could equally call it *method induction*. The novelty is **not** the induction but the *defeasible contract* that lets an induced method be un-learned. The change is control-flow: where a similarity memory does **query** → **retrieve** → **reuse**, ours does **query** → **retrieve-contract** → **assert** → **execute** → **verify** → **retract** → **specialize**. The verify-and-retract suffix — absent from retrieval, CBR, and skill libraries — is the difference, and what the experiments isolate.

Contributions. (1) The framing of reusable agent methods as **two-faced typed non-terminals with a defeasible runtime contract**, and the assume/assert/retract-blame/revise loop that un-learns on drift. (2) A reproducible, mechanism-isolating evaluation on a real engine — recall-only memory cannot un-learn while a declarative typed contract recovers drift *selectively, generally, composition-safely* (E2, stub + live, with a fair invalidating-cache baseline); sound structural transfer that call-count alone cannot certify (E1); no false-admits on the evaluated compositions, each of three gates ablated (E3); amortization as a gradient in the canonicalizable fraction (P4) — stating explicitly what each experiment does and does not establish.

2 Approach

2.1 The object: a two-faced method

A **method** is, to its caller, a single black box with a typed contract; inside, one or more *productions* realize it (sequence, branch, map, fold). Formally it is a hyperedge-replacement non-terminal [15, 9] with precondition-gated selection [12]. We keep two regimes apart *by design intent* (we do not prove decidability here): the **method grammar** (selection, parameterization, composition) is *intended* to stay decidable via a well-founded mount-rank and typing invariants — recursive HTN plan-existence is undecidable in general [13], which is why we restrict to the well-founded fragment — while **execution** over runtime-sized data is an explicitly fuel-bounded, Turing-complete layer. The connection to monadic-second-order definability [6] is motivation for why grammar-level checks *can* be tractable, not a theorem established here.

2.2 The contract: a defeasible separation triple

A method declares a read-footprint, a write-footprint, a precondition over what it reads, a postcondition over what it writes, and an effect tag. Composition under shared state is the frame problem [17]; we discharge it with a separation-logic footprint discipline [18, 22] over the finite typed alphabet. For a *learned* method the postcondition is an induced hypothesis: **assumed at compose time, asserted at settle time, retracted with blame on violation** [14, 3]. The runtime monitor is the JTMS [8], giving *eventual* (not static) soundness — exactly the un-learning the baselines lack.

2.3 Pipeline and the K1 floor

A formulation is typed into a goal; a method is selected and composed on contracts; cases flow through it; traces distil (anti-unify [21]; MDL-gated [11, 4]) into new typed methods; drift retracts. The universal fallback is the *micro-task floor*: anything that does not reduce to a cached typed method reduces to a micro-task a small model does easily. A *missing* contract costs a cheap model call (a graceful cost gradient); a *wrong* contract is caught by the runtime assert — the two failure modes degrade cost and trigger un-learning respectively, never silent error.

3 Implementation

All mechanisms are realized on an existing rule-driven, JTMS-coherent typed-fact graph engine with declarative concepts and forward-chaining stabilization, with no core change beyond one additive, backward-compatible query option. The typed memo key is the engine’s canonicalization digest; the defeasible-contract checker (compose-time entailment over abstract domains, a runtime post-assertion, three soundness gates) and the structural-transfer transform (relativize-on-store / bind-on-replay) are host libraries over the engine. A case executor that runs validated methods durably at scale is a known artifact (a workflow net [26] over a durable store, in the lineage of AWS Step Functions Distributed Map [2], Prefect’s content cache, and DBOS [25]); our belief view sits above it. The defeasible lifecycle — **assume** → **assert** → **verify** → **retract** → **specialize** — maps one-to-one onto the engine functions it calls:

```
select(goal): # ASSUME (compose time)
  M <- library.match(goal.typed_facts) # typed key; a miss falls to the micro-task floor
  assume M.contract # checkCompose: post(prev) |= pre(M); escalate, never false-admit

apply(M, case): # ASSERT + VERIFY (run time)
  key <- digest(case.typed_premise) # K1 canonical key
  if memo.has(key): return memo[key] # amortize a recurrent typed case
  out <- run(M, case) # else derive (model call / sub-graph)
  if not assertPost(M.contract, out): # post holds? + G1 frame-completeness + G2 effect-oracle
    quarantine(case); blame(M.contract) # never commit a bad output
  return
  memo[key] <- out; return out

on ingest(fact): # RETRACT + SPECIALIZE (drift)
  for e in memo s.t. e depends on fact: # JTMS: re-assert each affected post against the new fact
    if not satisfies(e.contract.post, e.facts + {fact}):
      retract(e); blame(e.contract) # un-learn: evict the invalidated entry + assign blame
  library.revise(blame): pre <- specialize(pre) # reviseOnBlame (CEGIS): narrow the pre, do not
  delete
```

The verify-and-retract suffix is the only part absent from a similarity memory, and it is exactly the part the experiments isolate.

4 Experiments

4.1 Setup

Experiments use the real engine’s mechanisms at two stated fidelities. **E1 and E3 instantiate the full engine** (graph + stabilization + JTMS): E1 mounts structural methods and transfers them via relativize/bind; E3 composes real concepts and compares the box-closed `checkCompose` verdict to the engine’s actual stabilization outcome. **E2, P4, and E5 isolate the relevant engine functions** — the canonicalization key (`digest`), the contract re-assertion (`satisfies`), the canonicalization barrier (`canonValue`) — driving them from a harness rather than the full loop, so the *mechanism* is measured without the scheduler. We do not claim E2 exercises the engine’s native JTMS retraction (it re-implements that retraction over the same real predicate). The model runs as a **deterministic stub** (a perfect oracle of the *current* rule given only what each arm’s prompt reveals — so all staleness/cost come from the arm’s mechanism, not model error) or as a **live** local model (`qwen36-q2-vram`). Every run is gated by a **harness self-test** (under the stub the naive arm must be perfect, else abort — a direct response to a prior 0/24 instrumentation bug). Stub results are deterministic. Seven arms share one interface: Naive, Long-context, RAG, CBR (typed key, no re-validation), Skill (prose), **Invalidating** (typed-key cache + a coarse hand-coded class-callback: an invalidation hook but no typed contract), and **Struct** (the typed library with the defeasible contract). The Invalidating arm exists to separate “has an invalidation mechanism” from “has a typed defeasible contract.”

4.2 E2 — defeasance on drift

A typed approval domain ($N=80$, two audited classes; the live run uses $N=48$, one audited class) with an *external* mid-stream premise-invalidation: a compliance audit marks a class non-compliant, flipping its previously-approved cases to reject. The audit is **not a record field**, so a recall-only cache retrieves the same unchanged record and serves its cached pre-audit answer (Table 1).

arm	calls	overall acc	drift acc	max ctx
Struct (typed contract)	26	1.00	1.00	290
Invalidating (hook, no contract)	28	1.00	1.00	290
Naive	80	1.00	1.00	290
Long-context	80	1.00	1.00	2062
RAG	48	0.95	0.00	290
CBR (typed key, no re-validation)	24	0.95	0.00	290
Skill (prose)	80	0.95	0.00	297

Table 1: E2 (stub). Recall-only arms (RAG/CBR/Skill) serve stale; both Invalidating and Struct recover.

The reading is three-fold. **Recall-only memories serve stale** — recall alone cannot recover, because the audit never enters their reuse path. **Recovery requires an invalidation mechanism**, and both the Invalidating cache and Struct have one, so both reach drift-acc 1.00. What the **typed defeasible contract adds over the hand-coded callback** is (i) *selectivity* — Struct re-asserts

the post per entry (**satisfies**) and evicts only the 2 *violated* (approve) classes, where the callback coarsely drops whole classes (4 entries) and pays the extra re-derivations (26 vs 28 calls); (ii) *generality* — the same **assertPost** handles any premise; (iii) *composition-safety* (§4.4). The live run (**qwen36-q2-vram**, $N=48$) reproduces this: RAG/CBR/Skill drift-acc 0.00; Invalidating 14 calls / drift 1.00; Struct 13 calls / 2.8 s / drift 1.00 / ctx 278 vs Long-context 1304. The defensible claim is not “only Struct recovers” but “recall-only memory cannot un-learn, and a declarative typed contract provides the recovery selectively, generally, and composition-safely.”

4.3 E1 — amortization and structural transfer

A structural-decomposition domain (a method that *creates* a sub-graph with object ids), on the full engine. Split: train, held-out related (same typed transitions, fresh id-spaces), held-out novel. This is an existence-and-soundness check on a small set (2 held-out related, 1 novel), not a population rate: with the relativize/bind transform, all held-out related instances transfer at 0 calls and **sound**, the novel transition pays (no false replay), totals 3 calls vs the no-cache baseline’s 5. The no-transform ablation (a flat content cache) “hits” the related instances but replays the *wrong id-space* — **unsound**. The point is qualitative: a call-count-only metric ranks the flat cache equal to the transform (both elide), so *only the soundness check distinguishes sound reuse from a wrong-id-space replay*. Scaling to a transfer rate over many methods is future work.

4.4 E3 — composition soundness

Composing method pairs purely on their typed contracts (box closed) and comparing to the open-the-box engine outcome (full engine), the box-closed decision matches reality on every evaluated pair, with **no false-admits** — the checker never false-accepts; under-determined or out-of-fragment pairs *escalate* (to a micro-task) rather than admit. This is shown on a small, hand-constructed set (3 pairs spanning sound / unsound / escalate; a 4th adds the oracle case): an existence demonstration of soundness, not a population rate. Each of three gates is shown load-bearing on a dedicated example: removing frame-completeness misses an undeclared write; removing the effect-tag gate silently admits an unverified external effect; removing footprint-cycle detection admits a coupled-retractable cycle. The decision reads only the shared footprint, never the body. The checker itself (per-key abstract-domain entailment, sound-but-incomplete, escalate-on-doubt) is the most developed formal artifact and, if anything, under-evaluated here. A larger, non-hand-picked corpus is future work.

4.5 P4 — the K1-coverage ceiling

On a mixed workload (fraction p fully typed; the rest carrying a prose component that overrides the typed rule), with K1-membership decided by the real canonicalization barrier, amortization is a **gradient in coverage** (approval: $0 \rightarrow 19 \rightarrow 44 \rightarrow 69 \rightarrow 94\%$ elided at $p = 0/.25/.5/.75/1$; triage: $0 \rightarrow 22 \rightarrow 47 \rightarrow 72 \rightarrow 97\%$). Struct’s accuracy is 1.00 at every coverage — the non-typed fraction is a micro-task *cost*, never a soundness *cliff*. A greedy variant that memoizes prose-bearing records on their typed key drops to an accuracy equal to the clean fraction: amortizing beyond the canonicalizable fraction is unsound. We are explicit that this is a **constructed illustration**, not a real-workload measurement: we *set* p and *define* prose records to override the typed rule, so “amortizing past K1 is unsound” follows by construction. What it establishes is the *shape* (amortization proportional to coverage) and that soundness holds at every level; the canonicalizable fraction of a real corpus is domain-dependent and not measured here.

4.6 E5 — bookkeeping cost at stream scale

A bookkeeping-cost check, not a claim about scaling the hard part: over a 200-class typed space with one audit, as stream length N grows $1,320 \rightarrow 20,320$ (the class set is fixed; no new methods, no model), Table 2.

N	Struct calls	calls/ N	Naive calls	library (memo)	evicted on drift
1,320	202	0.153	1,320	200	2
5,320	202	0.038	5,320	200	2
20,320	202	0.010	20,320	200	2

Table 2: E5. Struct calls stay constant; library bounded by the (fixed) class set; eviction selective.

Struct’s call count stays constant (distinct classes plus drift re-derivations), so $\text{calls}/N \rightarrow 0$ while Naive stays at one; the library is bounded by the class count (trivially — a map over a bounded class set); a drift event retracts only the invalidated classes ($O(\text{invalidated})$: 2 of 200). Per-op costs are small: canonicalization $\approx 0.5\text{--}3.5\mu\text{s}/\text{call}$ (the spread is JIT warmup — $\sim 3.5\mu\text{s}$ cold, settling to $\sim 0.5\mu\text{s}$), one drift’s eviction pass $\approx 0.5\text{ms}$ over the library. What E5 establishes is narrow: the typed bookkeeping does not become the bottleneck as the stream grows. It does **not** test scaling in the dimension that matters — a growing library of *distinct* methods, a real corpus, or a live model across all arms — which we leave to future work.

4.7 E6 — head-to-head against named agent-memory systems

The Setup baselines are generic (RAG/CBR/Skill); the systems a reviewer compares against are the *named* ones. We add a faithful minimal re-implementation of MemGPT/Letta [20] (tiered self-editing memory), Reflexion [24] (failure-driven verbal reflection, no decision memo), and GraphRAG [10] (an offline community-summary index) behind the same interface, each in its *fairest* configuration with a paired ablation (the negative control). Deterministic stub, $N = 78$, two audited classes (Table 3).

arm	calls	acc	drift-acc	max ctx
Naive	78	1.00	1.00	290
MemGPT (audit paged into core)	23	1.00	1.00	320
–paging (ablation)	18	0.92	0.00	258
Reflexion (delayed failure signal)	82	1.00	1.00	332
–signal (ablation)	78	0.92	0.00	258
GraphRAG (offline index)	87	0.92	0.00	336
+re-index (ablation)	89	1.00	1.00	336
Struct	20	1.00	1.00	290

Table 3: E6. Given its fairest shot each named system can recover; only Struct is Pareto-optimal on (calls, drift-acc, context).

The honest reading is *not* “only Struct recovers”: given its fairest shot each named system can recover on drift. The claim is that Struct recovers at the lowest cost on all three axes at once — the **unique Pareto-optimal point** on (calls $\times$ correct-on-drift $\times$ per-call context); no other arm matches-or-beats it on all three. Each named system pays a mechanism-specific tax: MemGPT

recovers only by paging the surfaced audit into core memory (a model turn), then *coarsely* (a whole-class flag re-deciding even unaffected members) with a core block carried in every prompt; Reflexion has *no* content-addressed memo, so it issues a model call on *every* record (calls $\approx N$) and recovers only reactively after an observed failure; GraphRAG’s offline index is blind to the silent audit (stale), recovering only by a *batch* re-summarization (coarse, off-band), never a per-entry defeater. The ablations confirm each mechanism is load-bearing (off $\rightarrow$ drift-acc 0.00). In isolation, Struct’s recovery tax is the re-derivation of only the *violated* entries (selective) and the contract re-assertion is in-engine (**zero** model calls). A live run (**qwen36-q2-vram**, $N = 32$) reproduces the ordering: Struct 9 calls / 1.9s, MemGPT 11 / 2.3s, Reflexion 34 / 7.1s, GraphRAG 36 / 7.7s (stale). These are minimal faithful re-implementations, not the full systems.

5 Related Work

Retrieval and case memory. RAG [16] and CBR recall by similarity and reuse-or-adapt; they cannot represent a *premise becoming invalid*, so an exogenous change that leaves the query unchanged leaves the cached answer retrievable and stale (E2). Skill libraries (Voyager [27]) store *prose* skills with no typed defeasible premise; a stale skill stays applicable and must be re-applied by the model.

LLM agent memory. MemGPT/Letta’s tiered virtual context [20], Reflexion’s episodic buffer [24], and graph-structured retrieval (GraphRAG [10]) manage *what to keep and recall* cleverly, but recall by relevance/recency/similarity; to our knowledge none represent a typed premise whose falsification retracts a prior reuse. They are complementary: a defeasible contract could sit beneath any of them as the retraction layer. We run this head-to-head in E6 (§4.7): each, given its fairest shot, can recover on drift, but only the defeasible contract does so at the lowest cost on (calls $\times$ correctness $\times$ context) simultaneously.

Library learning / EBL. DreamCoder [11] and Stitch [4] grow a library by abstraction [21]; these learn *what* to reuse but attach no defeasible runtime contract.

Contracts, blame, gradual verification. Higher-order contracts with blame [14] and gradual/hybrid verification [3] are the lineage of our assume/assert/retract-blame discipline, applied to *learned* methods.

Composition: graph grammars, HTN, separation logic. A two-faced HRG non-terminal [15, 9, 6] with HTN precondition-gated selection [12]; recursive HTN plan-existence is undecidable [13]. Sound composition under shared state is the frame problem [17], discharged by a separation-logic footprint discipline [18, 22] (E3).

Theory revision and belief revision (the nearest neighbor). **reviseOnBlame** — specialize a learned precondition from a counterexample rather than delete the method — is theory revision of a *learned* rule base: EITHER [19] and FORTE [23] revise Horn-clause theories on contradicting examples, exactly our blame $\rightarrow$ specialize step; the contraction/revision of a belief set is AGM [1]. We claim no new revision operator; our contribution relative to this line is operational — attaching the revision to a *typed, composable, canonicalizable method library* with a runtime contract, so amortization, composition-checking, and un-learning share one representation.

Truth maintenance. The un-learn mechanism is a JTMS [8, 7].

Durable execution / IVM. A case is a 1-safe marking over a workflow net [26]; the durable executor is a known artifact [2, 25]. DBSP [5]/Materialize maintain *values* incrementally, and production caches invalidate on source change — both are, in effect, the “invalidation hook” our fair baseline models; we claim no novelty over them for *recovering* on drift. The narrower difference: our invalidation is *derived from a declarative contract* (re-assert the post; blame; specialize), so it

generalizes across premises and feeds library revision.

6 Threats to Validity

Stub vs. live. The deterministic stub removes model error to isolate each arm’s *mechanism*; the live E2 confirms the ordering with a real model. The stub does not predict absolute live accuracy; running more arms live is future work.

K1-coverage is parameterized. P4 *sets* the typed fraction; the non-circular claims are the *shape* (a gradient), universality of soundness, and that greedy amortization is unsound. The absolute canonicalizable fraction of a production workload is domain-dependent.

Baseline strength. RAG/CBR/Skill are our implementations; the Invalidating baseline isolates “has an invalidation hook” from “has a typed contract,” so the claim is precise: recall-only memory *cannot* recover, and a declarative typed contract recovers selectively/generally/composition-safely. A tuned event-invalidating RAG/CBR is essentially the Invalidating arm and should match Struct on drift-accuracy, differing on selectivity/generality. The *named* systems are evaluated directly in E6 (§4.7); they are minimal faithful re-implementations of each mechanism (no real LLM-driven memory edits, learned reward model, or Leiden communities), each granted its fairest, *charitable* configuration — upper bounds on its drift behavior, not strawmen. Symmetrically, our STRUCT is not a stub: a real-engine realization (ensure-gated JTMS retraction over the derivation cache) reproduces its corner stub and live (9 calls on the live $N = 32$) and the warm library survives a restart at zero model calls — E6 compares the named-system re-implementations against the *actual* engine, not stub-against-stub.

Novelty / positioning. No mechanism is new; the work is a *composition* of JTMS, contracts-with-blame, library learning, theory revision, and separation-logic footprints. We position it as an operational unification, not a new algorithm.

Scale and breadth. Mechanism experiments use modest N over two synthetic domains; E5 is bookkeeping cost. The head-to-head vs modern agent-memory systems is now E6 (§4.7, minimal faithful re-implementations); a real corpus on a durable executor and a comparison vs the *deployed* systems remain future work.

Soundness is eventual, not static. Applicability of a learned method is undecidable in general (Rice); the guarantee is eventual soundness via a load-bearing runtime monitor over a sound-but-incomplete compose-time gate.

Single engine / author. All results are on one implementation; independent reproduction would strengthen the structural claims.

7 Conclusion

Recall-only agent memory cannot un-learn; static libraries are sound but frozen. A learned library of typed methods with a **defeasible runtime contract** gets both: it amortizes and composes on typed structure, and when a premise drifts it *retracts with blame and revises* — recovering correctness where retrieval, CBR, and skill libraries serve stale. Our experiments isolate this: recall alone cannot recover; an invalidation hook can; a *declarative typed contract* provides that recovery selectively, generally, and composition-safely, at bounded per-call context, on a real engine and confirmed live — bounded by a canonicalizable fraction that is itself a soundness boundary. Every mechanism is prior art; the contribution is their composition into a single typed representation in which amortization, composition-checking, and un-learning coincide. It is, deliberately, an engineering synthesis with a

testable emergent property — principled, selective un-learning — rather than a new algorithm; we think that property is worth naming and measuring.

Code & reproducibility. The engine and a self-contained experiment artifact are public at github.com/9pings/skynet-graph — artifact/paper-dll/ (workload/arms/harness + **e1-transfer**, **e3-compose**, **p4-coverage**, **scale**, **measure-e2-live**, **F6-transfer**) with the deterministic suite `tests/integration/paper-*.test.js` (npm test). The live E2 uses a local OpenAI-compatible endpoint (**qwen36-q2-vram**). Licensed AGPL-3.0-or-later.

References

- [1] C. E. Alchourrón, P. Gärdenfors, D. Makinson. On the Logic of Theory Change. *J. Symbolic Logic* 50(2):510–530, 1985.
- [2] AWS. Step Functions Distributed Map. 2022.
- [3] J. Bader, J. Aldrich, É. Tanter. Gradual Program Verification. *VMCAI* 2018, LNCS 10747:25–46.
- [4] M. Bowers et al. Top-Down Synthesis for Library Learning. *POPL* 2023; PACMPL 7(POPL).
- [5] M. Budiu et al. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *PVLDB* 16(7):1601–1614, 2023.
- [6] B. Courcelle. The Monadic Second-Order Logic of Graphs I. *Inf. Comput.* 85(1):12–75, 1990.
- [7] J. de Kleer. An Assumption-based TMS. *Artificial Intelligence* 28(2):127–162, 1986.
- [8] J. Doyle. A Truth Maintenance System. *Artificial Intelligence* 12(3):231–272, 1979.
- [9] F. Drewes, H.-J. Kreowski, A. Habel. Hyperedge Replacement Graph Grammars. In *Handbook of Graph Grammars* Vol. 1, World Scientific, 95–162, 1997.
- [10] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.
- [11] K. Ellis et al. DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning. *PLDI* 2021.
- [12] K. Erol, J. Hendler, D. S. Nau. UMCP: A Sound and Complete Procedure for HTN Planning. *AIPS* 1994.
- [13] K. Erol, J. Hendler, D. S. Nau. Complexity Results for HTN Planning. *Ann. Math. AI* 18:69–93, 1996.
- [14] R. B. Findler, M. Felleisen. Contracts for Higher-Order Functions. *ICFP* 2002, 48–59.
- [15] A. Habel. Hyperedge Replacement: Grammars and Languages. LNCS 643, Springer, 1992.
- [16] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* 2020.

- [17] J. McCarthy, P. J. Hayes. Some Philosophical Problems from the Standpoint of AI. *Machine Intelligence* 4, 1969.
- [18] P. W. O’Hearn, J. C. Reynolds, H. Yang. Local Reasoning about Programs that Alter Data Structures. *CSL 2001*, LNCS 2142:1–19.
- [19] D. Ourston, R. J. Mooney. Theory Refinement Combining Analytical and Empirical Methods. *Artificial Intelligence* 66(2):273–309, 1994.
- [20] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [21] G. D. Plotkin. A Note on Inductive Generalization. *Machine Intelligence* 5:153–163, 1970.
- [22] J. C. Reynolds. Separation Logic: A Logic for Shared Mutable Data Structures. *LICS 2002*, 55–74.
- [23] B. L. Richards, R. J. Mooney. Automated Refinement of First-Order Horn-Clause Domain Theories. *Machine Learning* 19(2):95–131, 1995.
- [24] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS* 2023.
- [25] A. Skiadopoulos et al. DBOS: A DBMS-oriented Operating System. *PVLDB* 15(1):21–30, 2022.
- [26] W. M. P. van der Aalst. The Application of Petri Nets to Workflow Management. *J. Circuits, Systems and Computers* 8(1):21–66, 1998.
- [27] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.